

EECS 836 Homework 1

JOHNPAUL MBAGWU

1

Problem 1: Linear Regression I

We are given the optimization problem:

$$w^* = \arg \min_w \left(\frac{1}{2} \sum_{i=1}^n (y_i - wx_i)^2 + \lambda w^2 \right) \quad (1)$$

where $\lambda \geq 0$ is a regularization parameter.

Step 1: Expand the Objective Function

Expanding the squared term:

$$J(w) = \frac{1}{2} \sum_{i=1}^n (y_i^2 - 2wx_iy_i + w^2x_i^2) + \lambda w^2 \quad (2)$$

Distribute $\frac{1}{2}$ across all terms:

$$J(w) = \frac{1}{2} \sum_{i=1}^n y_i^2 - \frac{1}{2} \sum_{i=1}^n 2wx_iy_i + \frac{1}{2} \sum_{i=1}^n w^2x_i^2 + \frac{1}{2}(2\lambda w^2) \quad (3)$$

Since $\frac{1}{2} \times 2 = 1$, this simplifies to:

$$J(w) = \frac{1}{2} \sum_{i=1}^n y_i^2 - w \sum_{i=1}^n x_iy_i + \frac{1}{2}w^2 \sum_{i=1}^n x_i^2 + \lambda w^2 \quad (4)$$

Step 2: Compute the Derivative

Taking the derivative with respect to w :

$$\frac{dJ}{dw} = - \sum_{i=1}^n x_iy_i + w \sum_{i=1}^n x_i^2 + 2\lambda w \quad (5)$$

Step 3: Solve for w^*

Setting $\frac{dJ}{dw} = 0$:

$$w \left(\sum_{i=1}^n x_i^2 + 2\lambda \right) = \sum_{i=1}^n x_iy_i \quad (6)$$

Solving for w^* :

$$w^* = \frac{\sum_{i=1}^n x_i y_i}{\sum_{i=1}^n x_i^2 + 2\lambda} \quad (7)$$

Conclusion

The optimal value of w in regularized linear regression is:

$$w^* = \frac{\sum_{i=1}^n x_i y_i}{\sum_{i=1}^n x_i^2 + 2\lambda} \quad (8)$$

This result shows that as λ increases, the denominator increases, which causes w^* to shrink toward zero, helping to prevent overfitting.

(b) Effect of λ on w^*

When $\lambda = 0$, we get the ordinary least squares (OLS) estimate:

$$w^* = \frac{\sum_{i=1}^n x(i)y(i)}{\sum_{i=1}^n x(i)^2} \quad (9)$$

As $\lambda \rightarrow \infty$, the denominator grows larger, making w^* approach zero. This means increasing λ causes more regularization, shrinking w^* toward zero.

Problem 2: Linear Regression II

Given the model:

$$Y = w_1 k + w_2 m \quad (10)$$

where k is the number of keyboards sold and m is the number of mice sold.

(a) Compute w_1 and w_2 with 101 observations

Given: - 100 observations: (1, 1, 1) - 1 observation: (1, 0, 1) Using Scikit-learn's

`LinearRegression(fit_intercept=False)`, we obtain:

$$w_1 = 1, \quad w_2 = 0 \quad (11)$$

(b) Adding two new observations

- 100: (1, 1, 1) - 1: (1, 0, 1) - 2: (0, 1, 1) The new weights change drastically due to instability.

(c) Reason for Fluctuations

- High multicollinearity makes small data changes significantly impact weights. - To test for instability, check the condition number of $X^T X$.

This problem arises because, with mostly homogeneous data (nearly all 1 keyboard, 1 mouse, 1 computer), the inputs are highly correlated. This high correlation leads to instability in the fitted regression parameters. Even a small change in the observations

can cause the estimated w_1 and w_2 to change drastically, as demonstrated by the transition from all 1s to adding 0s. In a real-world model, we would expect the true underlying parameters to be more stable representations of the relationships.

To check if a dataset may encounter this issue, we can:

- Assess the variability (variance) of each input feature—low variance indicates potential instability.
- Compute the correlation matrix between input features—high correlation leads to unstable and unpredictable parameter estimates.
- Calculate the condition number of the input matrix X —higher values signify more sensitivity to data changes.
- Visually plot and examine the data—lack of variability in inputs can be detected through plots.

By checking these metrics on a candidate dataset, we can detect possible issues with regression stability before even fitting a model. This allows us to preemptively handle datasets that may lead to jumping or unstable coefficient values. Remedial actions such as adding regularization or removing highly correlated features can help mitigate these issues. Overall, inspecting the collinearity and variability of the inputs is crucial to avoiding wildly fluctuating parameter estimates.

Problem 3: Logistic Regression

Given the decision function:

$$z = 4x_1 + 6x_2 - 24 \quad (12)$$

where the probability is given by:

$$p(y = 1|x) = \frac{1}{1 + e^{-z}} \quad (13)$$

(a) Compute $z(A)$ and $p(y = 1|A)$

For point $A = (3, 3)$:

$$z(A) = 4(3) + 6(3) - 24 = 6 \quad (14)$$

Computing the probability of point A being a positive example using the sigmoid function, we have:

$$p(y = 1|A) = \frac{1}{1 + e^{-x}} \quad (15)$$

$$p(y = 1|A) = \frac{1}{1 + e^{-6}} \quad (16)$$

$$p(y = 1|A) = \frac{1}{1 + 0.00248} \quad (17)$$

$$p(y = 1|A) = \frac{1}{1.00248} \quad (18)$$

$$p(y = 1|A) = 0.9975 \quad (19)$$

(b) Decision Boundary

To draw the decision boundary, we set the decision function $z(x_1, x_2) = 0$.

From the decision function

$$z(x_1, x_2) = 4x_1 + 6x_2 - 24$$

we solve for x_2 .

$$4x_1 + 6x_2 - 24 = 0$$

$$4x_1 + 6x_2 = 24$$

$$6x_2 = -4x_1 + 24$$

$$x_2 = \frac{-4x_1 + 24}{6}$$

$$x_2 = \frac{-2x_1}{3} + 4$$

When $x_1 = 0$, $x_2 = 4$ and when $x_2 = 0$, $x_1 = 6$. See Fig. 1 and Fig. 2.

(c) Compute accuracy

From the decision boundary, we can see that:

TP = 6	FN = 1
FP = 2	TN = 6

Where TP = True Positive; TN = True Negative; FP = False Positive; FN = False Negative.

Accuracy of the linear classifier is computed as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Accuracy} = \frac{6 + 6}{6 + 6 + 2 + 1} = \frac{12}{15} = 0.8 = 80\%$$

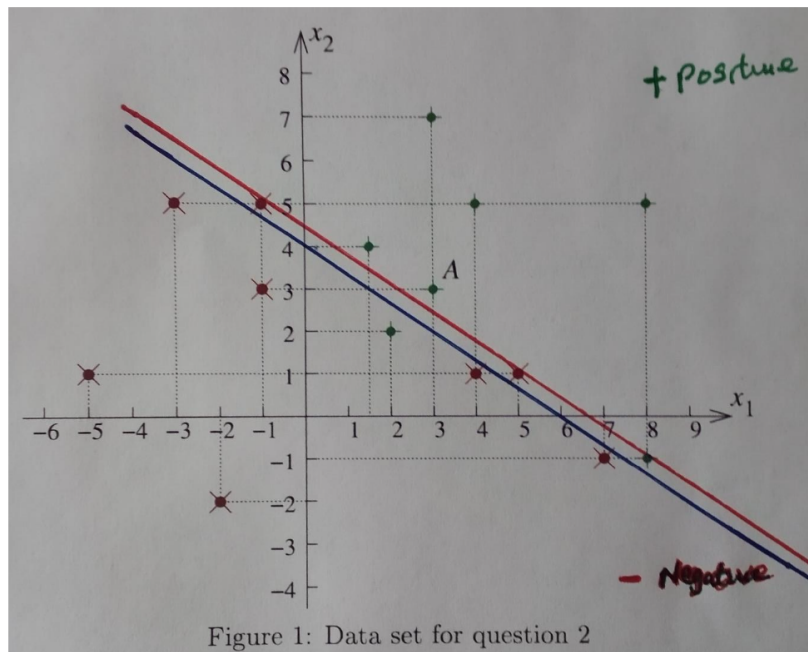


Figura 1. shows the decision boundary with a blue line, and it passes through the coordinates $(0, 4)$ and $(6, 0)$. The $(+)$ positive region is represented with the function $f(x_1, x_2) > 0$, and the $(-)$ negative region is represented with the function $f(x_1, x_2) < 0$, representing the upper and lower regions of the graph, respectively. Additionally, the red line represents the linear decision boundary that improves upon the accuracy. Figure 2 explicitly represents the blue line of the original decision boundary before the accuracy improvement was applied.

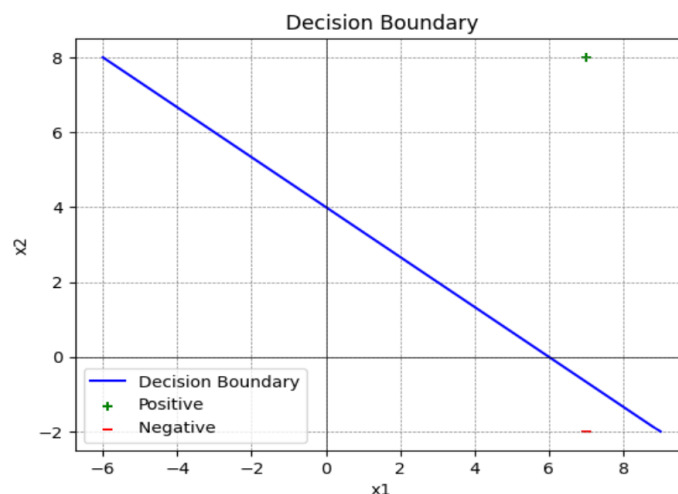


Figura 2. shows the decision boundary with a blue line, and it passes through the coordinates $(0, 4)$ and $(6, 0)$. The positive region is represented with a function $f(x_1, x_2) > 0$, and the $(-)$ negative region is represented with the function $f(x_1, x_2) < 0$, representing the upper and lower regions of the graph, respectively.

(d) Improve the classifier

Finding a decision border that more effectively divides the dataset's classes is usually our goal when trying to increase the accuracy of a linear decision function. Using a linear decision function with coefficients designed to maximize a certain performance metric or minimize misclassifications is one method to accomplish this.

Coordinates of the linear decision boundary that improves upon the accuracy are $(x_1, x_2) = (6.6, 0)$ and $(x_1, x_2) = (0, 4.5)$.

Applying the equation of a line, we have:

$$\frac{x_1}{6.6} + \frac{x_2}{4.5} = 1$$

$$4.5x_1 + 6.6x_2 = 29.7$$

$$4.5x_1 + 6.6x_2 - 29.7 = 0$$

From the given graph, the new decision boundary equation is:

$$z'(x_1, x_2) = 4.5x_1 + 6.6x_2 - 29.7 = 0$$

Here, the previous line is shifted up by 0.5 units.

Considering the red linear line on the graph that improves upon the accuracy, we can see that the accuracy improved by an additional 6.7%, resulting in an improved accuracy of 86.7% instead of the original 80%.

TP = 5	FN = 2
FP = 0	TN = 8

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Accuracy} = \frac{5 + 8}{5 + 8 + 0 + 2} = \frac{13}{15} = 0.867 = 86.7\%$$

Problem (4a) What are the possible reasons for model overfitting?

1. **Excessive variance in the training data:** High variance in the data can cause the model to fit the noise or random fluctuations rather than the underlying patterns, which leads to overfitting.

2. **Excessive noise in the training data:** Noise refers to random fluctuations or errors in the data that do not reflect the true relationship between the features and the target variable. If the training data contains a significant amount of noise, the model may overfit by learning to capture the noise instead of the actual pattern.

3. **Insufficient regularization:** Without adequate regularization techniques, such as L1 or L2 regularization, complex models may not be penalized enough, which allows them to overfit the data.

4. **Limited size of the training data:** A small training dataset may prevent the model from generalizing well to unseen data because it hasn't been exposed to enough information to capture the underlying patterns.

5. **High model complexity:** If the model is too complex for the amount of available training data, it may overfit by capturing noise or random fluctuations in the data instead of the true patterns. This is especially common when the model has too many parameters or features relative to the dataset size.

Problem 4(b): Avoiding Overfitting in Logistic Regression

Overfitting in logistic regression happens when the model learns noise in the training data instead of the actual pattern. Here are ways to prevent it:

Regularization (L1 and L2 Penalties)

- **L2 Regularization (Ridge Regression):** Adds a penalty term $\lambda \sum w^2$ to the loss function, preventing large coefficients and reducing overfitting.
- **L1 Regularization (Lasso Regression):** Adds a penalty term $\lambda \sum |w|$, which can shrink some coefficients to zero, effectively performing feature selection.

Reducing Model Complexity

- Avoid using too many features, especially if they are highly correlated.
- Perform feature selection or use Principal Component Analysis (PCA) to reduce dimensionality.

More Training Data

- If possible, collect more training data to make the model generalize better.
- Data augmentation (for image/text tasks) can help increase dataset variability.

Early Stopping

- Monitor performance on a validation set and stop training when performance starts to degrade.
- Useful in gradient-based optimization methods like stochastic gradient descent.

Cross-Validation

- Use k-fold cross-validation to ensure the model performs well on unseen data.

Using a Validation Set

- Helps in tuning hyperparameters and detecting overfitting before final evaluation.

Problem (4C) What's the difference between validation dataset and test dataset?

In machine learning, the **validation dataset** and the **test dataset** serve different purposes:

1. Validation Dataset:

- **Purpose:** Used during the training process to tune model hyperparameters and make decisions about model architecture.
- **Role:** Helps to prevent overfitting by providing an unbiased evaluation of the model's performance on data it hasn't seen during training. It is used to adjust and select the model's parameters (such as the number of layers, learning rate, regularization techniques, etc.).
- **Usage:** Typically used to tune the model, select the best version of the model, and make decisions about further training.
- **Timing:** After training each epoch but before testing the model on unseen data (test dataset).

2. Test Dataset:

- **Purpose:** Used to evaluate the final model's performance after training and hyperparameter tuning are complete.
- **Role:** Provides an unbiased evaluation of how the model performs on completely unseen data. It is used to estimate how the model will perform in real-world scenarios.
- **Usage:** No further changes to the model are made after testing; it is used to assess the generalization ability of the trained model.
- **Timing:** Only used after training and validation are complete, when the model is finalized.

In summary:

- **Validation dataset:** Used for model tuning and hyperparameter selection during training.
- **Test dataset:** Used for final evaluation of the model's performance after training and validation are complete.

Problem (4d) Why accuracy can be a bad metric to evaluate your classifier?

What other metrics you can use?

Accuracy may not be an ideal metric for evaluating a classifier, particularly in imbalanced datasets where one class predominates. In such situations, a classifier that always predicts the majority class might achieve high accuracy but fail to detect instances from the minority class. Additionally, accuracy does not offer insight into the types of errors made by the classifier. For instance, in a two-class problem with 9990 instances of class 0 and only 10 instances of class 1, a model that predicts all instances as class 0 would yield an accuracy of 99.9%.

However, this high accuracy is misleading, as the model would not detect any class 1 examples. Other metrics that can be considered are:

1. **Precision:** The proportion of true positive predictions among all positive predictions. It reflects the model's ability to correctly identify positive instances.

$$\text{Precision}(p) = \frac{TP}{TP + FP}$$

2. **Recall:** The proportion of true positive predictions among all actual positive instances. It measures the model's ability to capture all positive instances.

$$\text{Recall}(r) = \frac{TP}{TP + FN}$$

3. **F-measure (F1-score):** The harmonic mean of precision and recall, offering a balanced metric of the classifier's performance.

$$F - \text{measure} = \frac{2 \times (Precision \times Recall)}{Precision + Recall}$$

Alternatively,

$$F - \text{measure} = \frac{2 \times TP}{2 \times TP + FP + FN}$$

Problem (4e)

Let M represent the order of the polynomial.

1. In Figure 2(b), when a linear model ($M = 1$) is fit to data generated from the $\sin(2\pi x)$ function, the model does not provide a good fit. Here, $M = 1$ represents a first-order polynomial, which is too simplistic to accurately capture the behavior of the $\sin(2\pi x)$ function, resulting in poor fitting and underfitting. A linear model cannot capture the non-linear relationship between the input variable x and the target variable y .

2. In Figure 2(c), when a higher-order polynomial ($M = 9$) is used, the fit to the training data is excellent. However, this suggests a strong alignment with the training data, but it does not imply that the model will generalize well to unseen data. It may be considered overfitting, as the higher complexity polynomial model fits the noise in the data rather than the underlying pattern, leading to poor generalization to new data.